

Gitter - Complete Documentation

A comprehensive guide to the Gitter Git-like Version Control System

Built from scratch to understand Git internals

Generated from repository documentation

Table of Contents

- [Main Documentation](#)
 - [Architecture](#)
 - [Hasher Design](#)
 - [Tree Storage](#)
 - [Commit Implementation](#)
 - [Log Implementation](#)
 - [Reset Implementation](#)
 - [Checkout Implementation](#)
 - [Docker Usage](#)
 - [Code Coverage](#)
 - [Critical Behavior Gaps](#)
-



Gitter

A Minimal Git-like Version Control System Built with C++20

C++

20

license

MIT

tests

238 passing

Built from scratch to understand Git internals

A production-grade Git clone with Git-compatible object storage, branch management, and intelligent staging



Key Features



Core Functionality

- **Full Git Workflow** - Init, add, commit, status, log, checkout, reset
- **Git-Compatible Storage** - SHA-1 hashing, zlib compression, content-addressable objects
- **Branch Management** - Create, switch, and manage multiple branches
- **Intelligent Staging** - Fast size/mtime checks, glob patterns, auto-staging
- **Reliable & Robust** - Atomic writes, error recovery, input validation



Architecture Excellence

- 🧩 **Clean Design Patterns** - Command, Factory, Strategy, Facade, Singleton
- 📦 **Modular Architecture** - Separated CLI, Core, and Utility layers
- 🔒 **Type-Safe Error Handling** - `Expected<T>` monad pattern
- 🪄 **Comprehensive Testing** - 238+ tests with GoogleTest

📋 Quick Command Reference

Category	Command	Description
Repo	<code>gitter init [path]</code>	Initialize new repository
Stage	<code>gitter add <file></code>	Stage files/directories
Stage	<code>gitter add *.cpp</code>	Stage with glob patterns
Commit	<code>gitter commit -m "msg"</code>	Create commit with message
Commit	<code>gitter commit -am "msg"</code>	Auto-stage & commit
Info	<code>gitter status</code>	Show working tree status
Info	<code>gitter log</code>	Display commit history
Branch	<code>gitter checkout <branch></code>	Switch to branch
Branch	<code>gitter checkout -b <branch></code>	Create & switch branch
Undo	<code>gitter reset HEAD~1</code>	Reset to previous commit
Undo	<code>gitter restore --staged <file></code>	Unstage files
Inspect	<code>gitter cat-file blob <hash></code>	View object content

🎬 Getting Started

Prerequisites

Option 1: Docker (Easiest - No Build Required!)

- Docker & Docker Compose installed

Option 2: Build from Source

- **CMake 3.20+**
- **C++20 Compiler** (GCC 10+, Clang 12+, MSVC 2022+)
- **Linux/macOS/WSL** or **Windows**

Quick Start with Docker 🐳

```
# Build and run interactively with welcome message!
docker-compose build
docker-compose run --rm gitter
```

```
# Or run directly without docker-compose
docker build -t gitter-cli:latest .
docker run -it --rm gitter-cli:latest
```

[Full Docker instructions →](#)

Quick Build

```
# Configure and build
cmake --preset linux-debug
cmake --build --preset linux-debug-build

# Run it!
./build/linux-debug/gitter help
```

Quick Example

```
# Initialize repository
gitter init

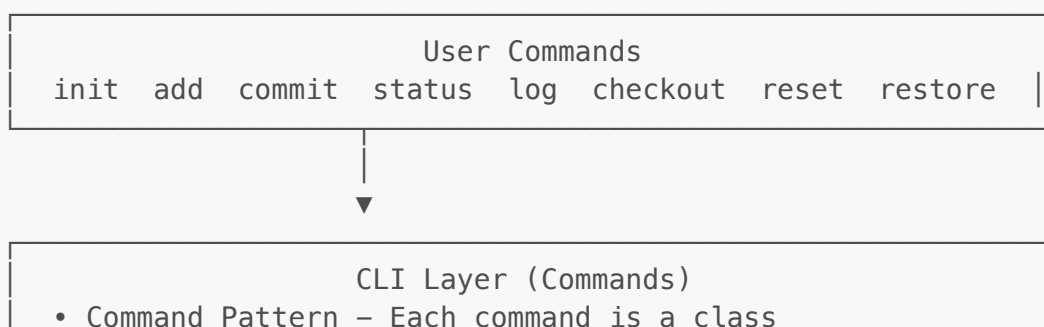
# Stage files
echo "Hello World" > file.txt
gitter add file.txt

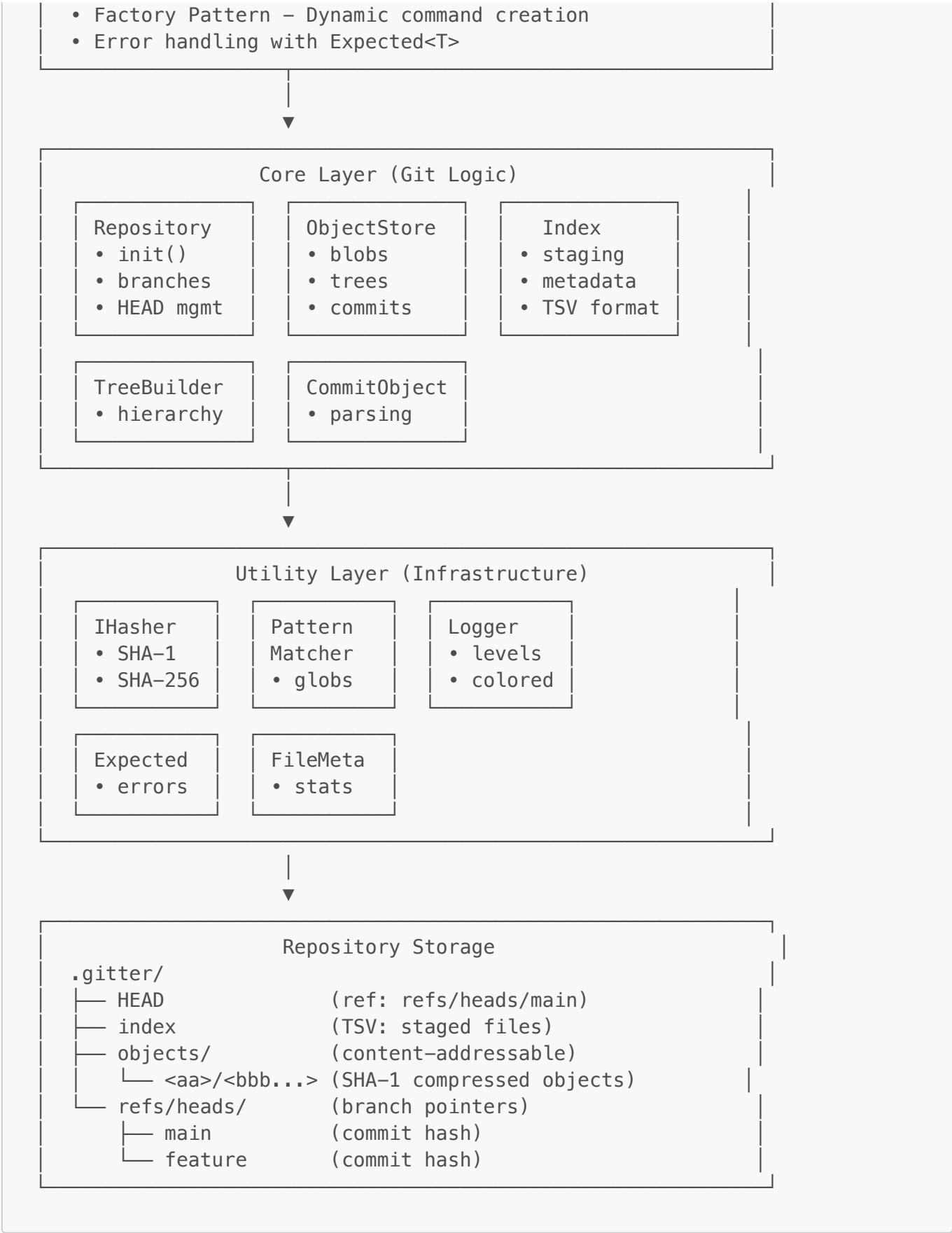
# Create commit
gitter commit -m "Initial commit"

# View history
gitter log

# Check status
gitter status
```

Architecture Overview





- **Atomic Writes:** Temp file pattern prevents index corruption
- **Content Deduplication:** Objects stored once by hash
- **Zlib Compression:** Reduces disk usage by ~90%

 Reliability Features

- **Error Recovery:** Graceful handling of corrupt data
- **Input Validation:** Hash format checking, numeric validation
- **Base64 Paths:** Supports filenames with special characters (TAB, newlines)
- **File Permissions:** Tracks executable bit (mode 0100755)

 Advanced Features

- **Glob Patterns:** `*.cpp`, `src/**/*.h`, `test?.py`
- **Auto-Staging:** `-a` flag stages tracked files automatically
- **Multi-Paragraph Commits:** Multiple `-m` flags create separate paragraphs
- **Smart Branch Switching:** Preserves staged files across branches
- **Three-Way Status:** Compares HEAD vs Index vs Working Tree

 Technical Details

Design Patterns

Pattern	Use Case	Benefits
Command	Each CLI command is a class	Queue, log, test commands independently
Factory	Dynamic command creation	Loose coupling between parsing and execution
Singleton	Repository, Logger, Factory	Single global instance for shared state
Strategy	Pluggable hashing (SHA-1/SHA-256)	Swap algorithms without changing clients
Facade	Repository hides complexity	Simple API for complex operations

Git Object Storage

Object Type	Format	Example
Blob	<code>blob <size>\0<content></code>	<code>blob 12\0Hello World</code>
Tree	<code>tree <size>\0<entries></code>	<code>tree 64\0100644 file.txt\0<hash></code>
Commit	<code>commit</code> <code><size>\0<metadata></code>	<code>commit</code> <code>234\0tree...\nparent...\nauthor...</code>

Storage Layout (Git-compatible):

```
.gitter/objects/  
└─ <first-2-chars>/      # Directory: "ab"  
    └─ <remaining-38>    # File: "c123def456..."
```

Compression: All objects zlib-compressed before storage

Index Format (TSV)

```
path<TAB>hash<TAB>size<TAB>mtime<TAB>mode<TAB>ctime
```

Example:

```
src/main.cpp<TAB>abc123...<TAB>1024<TAB>mtime<TAB>33188<TAB>ctime
```

(Note: Full timestamps in practice, shown simplified here)

Fields:

- **path** - Base64-encoded relative path (handles special characters)
- **hash** - SHA-1 blob hash (40 hex chars)
- **size** - File size in bytes
- **mtime** - Modification time (nanoseconds)
- **mode** - File permissions (0100644 = file, 0100755 = executable)
- **ctime** - Creation time (nanoseconds)



 Complete Build Guide

Platform Support

Platform	Compiler	Status
Linux	GCC 10+, Clang 12+	 Fully Supported
macOS	Clang 12+	 Fully Supported
Windows	MSVC 2022+*	 Fully Supported
WSL	GCC 10+	 Fully Supported

* **Windows Note:** Visual Studio 2022 requires the **Desktop Development with C++** workload installed. If you encounter "could not find any instance of Visual Studio", install this workload via Visual Studio Installer.

Build Commands

```
# Configure build
cmake --preset linux-debug

# Compile
cmake --build --preset linux-debug-build

# Build with tests
cmake --build build/linux-debug --target gitter_tests

# Release build
cmake --preset linux-release
cmake --build --preset linux-release-build

# macOS - Xcode Generator
cmake --preset macos-debug
cmake --build --preset macos-debug-build      # Build Debug configuration
cmake --build --preset macos-release-build    # Build Release
configuration

# macOS - Ninja Generator (Alternative)
cmake --preset macos-ninja-debug
cmake --build --preset macos-ninja-debug-build

# Visual Studio 2022 (Windows) - Requires Desktop Development with C++
workload
cmake --preset windows-vs2022
cmake --build --preset windows-vs2022-debug  # Build Debug configuration
cmake --build --preset windows-vs2022-release # Build Release
configuration
```

Alternative: Traditional CMake

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j4
./build/gitter help
```

Convenience: Using the Wrapper Script

After building, you can use the `gitter` wrapper script from the project root:

```
# The script automatically finds the first available built gitter
executable
./gitter help
./gitter status
./gitter add file.txt
./gitter commit -m "My changes"

# Or add to PATH for global access
```

```
export PATH="$PWD:$PATH"
gitter help
```

The wrapper searches build directories in this order:

1. `build/linux-release/gitter`
2. `build/linux-debug/gitter`
3. `build/macos-*/gitter` (all variants)
4. `build/windows-*/gitter.exe` (all variants)
5. Any other `build/*/gitter*` executables

Docker Support (No Build Required!)

Try Gitter instantly without installing dependencies:

```
# Build the image first
docker-compose build

# Run interactively with welcome message!
docker-compose run --rm gitter

# You're now in a full Linux Ubuntu bash shell with welcome message
displayed
# Demo project is in /demo directory
gitter help
cd /demo
gitter init demo-repo
cd demo-repo
echo "Hello World" > file.txt
gitter add file.txt
gitter commit -m "Initial commit"
gitter log

# Exit (container is automatically removed with --rm)
exit

# Or use Docker directly
docker build -t gitter-cli:latest .
docker run -it --rm gitter-cli:latest
```

See [Docker Usage Guide](#) for complete instructions and customization options.

Detailed Usage Examples

◆ Working with Files

► Stage Files - Single, multiple, directories, patterns


```
# Single file
gitter add file.txt

# Multiple files
gitter add file1.txt file2.cpp

# Directory (recursive)
gitter add src/

# Current directory (all files)
gitter add .

# Glob patterns
gitter add *.cpp           # All .cpp files
gitter add src/*.h         # All .h files in src/
gitter add test?.py        # test1.py, test2.py, etc.
gitter add **/*.json       # All .json files recursively
```

► Check Status - Three-way comparison

```
gitter status

# Output example:
# On branch main
#
# Changes to be committed:
#   modified:   file1.txt
#
# Changes not staged for commit:
#   modified:   file2.cpp
#
# Untracked files:
#   newfile.py
```

► Unstage Files - Remove from staging area

```
# Single file
gitter restore --staged file.txt

# Glob patterns
gitter restore --staged *.cpp
gitter restore --staged src/*.h
```

◆ Commits & History

► Create Commits - Single & multi-paragraph messages

```
# Basic commit
gitter commit -m "Fix bug in feature X"

# Auto-stage tracked files
gitter commit -am "Quick fix for bug"

# Multi-paragraph message
gitter commit -m "Summary line" -m "Detailed explanation" -m "Notes"
```

► View History - Commit logs

```
gitter log

# Shows last 10 commits with:
# - Commit hash (short)
# - Author & email
# - Date & timezone
# - Commit message
```

► Inspect Objects - View Git objects

```
# View blob content
gitter cat-file blob abc123def456...

# View tree entries
gitter cat-file tree def456ghi789...

# View commit metadata
gitter cat-file commit ghi789jkl012...

# Show object type
gitter cat-file -t abc123...

# Show object size
gitter cat-file -s abc123...
```

◆ Branches & History

► Branch Management - Create, switch, manage branches

```
# Switch to existing branch
gitter checkout feature

# Create and switch to new branch
gitter checkout -b new-branch
```

```
# Switch back to main
gitter checkout main
```

Note: Checkout preserves staged files across branches (Git-compatible)

► **Reset Commits** - Undo commits

```
# Reset to previous commit (moves HEAD back)
gitter reset HEAD~1

# Reset two commits back
gitter reset HEAD~2

# Reset to current HEAD (clear index)
gitter reset HEAD
```

Note: **reset** clears the index, leaving working tree files as untracked

How It Works

► **Command Implementation Details**

Add Command Flow

```
Pattern Detection → File Discovery → Hash Computation →
Object Storage (zlib) → Index Update
```

Status Command Flow

```
Three-Way Comparison:
├ Index vs HEAD → "Changes to be committed"
├ Working Tree vs Index (fast size/mtime check)
│   └ Hash only if changed → "Changes not staged"
└ Find untracked files → "Untracked files"
```

Commit Command Flow

```
Parse Args → Load Index → Auto-Stage (if -a) →
Build Tree (recursive) → Check Duplicates →
Create Commit Object → Update Branch Ref → Silent Success
```

Checkout Command Flow

```
Parse Args → Resolve Branch → Intelligent Index Merge →  
Restore Working Tree → Remove Files → Clean Dirs → Update HEAD
```

Smart Index Merging: Preserves staged uncommitted files across branches (Git-compatible)

► Error Handling & Reliability

Expected Pattern

Type-safe error propagation without exceptions:

```
Expected<void> result = command.execute(ctx, args);  
if (!result) {  
    std::cerr << result.error().message << "\n";  
    return 1;  
}
```

Error Codes

- **InvalidArgs** - Invalid command arguments
- **NotARepository** - Not in a git repository
- **IoError** - File I/O failures
- **AlreadyInitialized** - Repo already exists
- **CorruptObject** - Corrupted object data
- **RefNotFound** - Branch/commit not found
- **EmptyIndex** - Nothing to commit

Reliability Features

-  **Atomic Index Writes:** Temp file + rename pattern
-  **File I/O Validation:** All writes verified for success
-  **Input Validation:** Hash format checking
-  **Graceful Recovery:** Skips corrupted entries
-  **Path Normalization:** Consistent paths prevent duplicates

► Logging System

```
# Set debug level  
export GITTER_LOG=debug  
  
# Run command  
gitter add file.txt  
# [debug] Executing command: add  
# [info ] Staging file: file.txt  
# [info ] Created blob: abc123...
```

Log Levels: **error**, **warn**, **info**, **debug**

Testing

Test Coverage: **238 Tests** 

Category	Count	Coverage
Unit Tests	~85	Core components (Index, ObjectStore, TreeBuilder)
Command Tests	~115	All CLI commands with edge cases
Integration Tests	~89	Complete Git workflows
Total	238	87-93% line coverage

Running Tests

```
# Build and run all tests
cmake --build build/linux-debug --target gitter_tests
./build/linux-debug/gitter_tests

# Verbose output
./build/linux-debug/gitter_tests --gtest_brief=1

# Or use CTest
cd build/linux-debug
ctest --output-on-failure --verbose
```

Test Highlights

-  **Add Command:** 15+ tests (patterns, special chars, errors)
-  **Commit Command:** 12+ tests (multi-message, auto-stage, duplicates)
-  **Status Command:** 8+ tests (three-way comparison)
-  **Log Command:** 18+ tests (traversal, formatting)
-  **Checkout Command:** 23 tests (branch switching, file preservation, directory cleanup)
-  **Reset Command:** 6+ tests (HEAD~n syntax, chain traversal)
-  **Integration:** 30+ workflow tests

Coverage Reports

See [docs/COVERAGE.md](#) for detailed coverage analysis.

Roadmap

Status: **Core Features Complete** 

Feature	Status	Notes
Basic VCS	✔ Complete	Init, add, commit, status, log
Branch Management	✔ Complete	Create, switch, checkout
History Management	✔ Complete	Reset HEAD~n, traverse chains
Object Storage	✔ Complete	Blobs, trees, commits (Git-compatible)
Pattern Matching	✔ Complete	Glob patterns, wildcards
Reliability	✔ Complete	Atomic writes, error recovery
Advanced Features	🚧 Planned	Diff, merge, remotes, tags

Current Features ✔

- ✔ Full Git workflow (init, add, commit, status, log)
- ✔ Branch management (create, switch, checkout)
- ✔ History traversal (reset HEAD~n)
- ✔ Git-compatible storage (SHA-1, zlib, content-addressed)
- ✔ Intelligent staging (fast size/mtime checks)
- ✔ Pattern matching (.cpp, src/**/*.h)
- ✔ Error recovery (atomic writes, validation)
- ✔ 238 comprehensive tests

Next Steps 🚧

- ❑ **Diff Output** - Show file changes between commits
- ❑ **Merge Commits** - Basic three-way merge
- ❑ **Remote Operations** - push/pull/fetch
- ❑ **Tags** - Annotated tags for releases
- ❑ **Advanced Branch Ops** - Delete, rename branches

 Documentation

Architecture & Design

- [ARCHITECTURE.md](#) - System architecture, design patterns, data flow
- [HASHER_DESIGN.md](#) - Strategy pattern for hash algorithms
- [TREE_STORAGE.md](#) - How Git stores directory trees

Implementation Guides

- [COMMIT_IMPLEMENTATION.md](#) - Commit object creation
- [LOG_IMPLEMENTATION.md](#) - Commit history traversal
- [RESET_IMPLEMENTATION.md](#) - Reset command design
- [CHECKOUT_SUMMARY.md](#) - Branch switching and checkout

Testing & Quality

- [COVERAGE.md](#) - Test coverage analysis (238 tests)

Deployment & Distribution

- [DOCKER_USAGE.md](#) - Run Gitter with Docker (no build required!)

Code Quality

- **Doxygen Comments** - All public APIs documented
 - **Inline Documentation** - Complex logic explained
 - **Design Rationale** - Decisions documented in code
-

Learning Resources

Git Internals

- [Git Internals - Git Objects](#)
- [Git Index Format](#)
- [Pro Git Book](#)

Design Patterns

- Command Pattern for CLI architecture
 - Strategy Pattern for extensible hashing
 - Factory Pattern for dynamic creation
-

Troubleshooting

CMake Cache Issues

If you see errors like:

```
CMake Error: The current CMakeCache.txt directory ... is different than
the directory ...
```

Solution: Delete the old build directory and reconfigure:

```
rm -rf build/linux-debug build/linux-release
cmake --preset linux-debug
cmake --build --preset linux-debug-build
```

This happens when the project is moved or copied to a different location, as CMake stores absolute paths in the cache.

Build Fails

Windows: Ensure you have Visual Studio 2022 with "Desktop Development with C++" workload, or use WSL.

macOS: Install Xcode Command Line Tools: `xcode-select --install`

Linux: Install build essentials: `sudo apt install build-essential cmake ninja-build`

Tests Fail

Some tests require timing differences. If tests are flaky, add small delays or run them individually.

 License

Educational Project - Feel free to use, modify, and learn from this codebase.

Built with  to understand Git internals

Stars appreciated if this project helped you learn! 

Architecture

Directory Structure

```
src/
├── cli/                                # Command-line interface layer
│   ├── ICommand.hpp                  # Command interface
│   ├── CommandFactory.*              # Factory pattern for command creation
│   ├── CommandInvoker.*              # Command executor with error handling
│   └── commands/                     # Individual command implementations
│       ├── HelpCommand.*             #  Fully implemented
│       ├── InitCommand.*             #  Fully implemented
│       ├── AddCommand.*              #  Fully implemented (with glob patterns)
│       ├── CommitCommand.*           #  Fully implemented
│       ├── LogCommand.*              #  Fully implemented
│       └── StatusCommand.*           #  Fully implemented (with three-way
├── comparison)
│   ├── RestoreCommand.*              #  Fully implemented (with glob patterns)
│   ├── ResetCommand.*               #  Fully implemented (with HEAD~n syntax)
│   └── CheckoutCommand.*             #  Fully implemented (branch switching and
└── creation)
    ├── core/                         # Git core logic
    │   ├── Repository.*              # Repository management (Singleton)
    │   └── ObjectStore.*             # Git object storage (blobs/trees/commits)
```


├─ Index.*	# Staging area management
├─ TreeBuilder.*	# Builds tree objects from index
├─ CommitObject.hpp	# Commit metadata structure
└─ Constants.hpp	# Centralized constants (SHA-1 length, etc.)
└─ util/	# General-purpose utilities
├─ Logger.*	# Leveled logging system
├─ Expected.hpp	# Result/error handling
├─ IHasher.hpp	# Hash algorithm interface (Strategy Pattern)
├─ HasherFactory.*	# Factory for creating hasher instances
├─ Sha1Hasher.*	# SHA-1 implementation (Git default)
├─ Sha256Hasher.*	# SHA-256 implementation (Git future)
└─ PatternMatcher.*	# Glob/regex pattern matching

Layer Separation

1. Utility Layer (`src/util/`)

Purpose: Reusable, generic utilities with no Git-specific logic

- **IHasher / HasherFactory** - Cryptographic hashing (Strategy Pattern)
 - Pure utility, no domain knowledge
 - Supports SHA-1 (Git default) and SHA-256
 - Factory creates hasher instances by name
 - Used by ObjectStore for content addressing
- **PatternMatcher** - Glob pattern to regex conversion
 - Generic file pattern matching
 - Used by Add and Restore commands
- **Logger** - Leveled logging (error/warn/info/debug)
 - Environment-controlled via `GITTER_LOG`
 - Used throughout for diagnostics
- **Expected** - Result/error monad pattern
 - Type-safe error propagation
 - Avoids exceptions for control flow

2. Core Layer (`src/core/`)

Purpose: Git-specific domain logic and data structures

- **Repository** - Repository lifecycle and discovery
 - Singleton pattern for global state
 - Initializes `.gitter/` structure
 - Discovers repo root by walking up directories

- **HEAD management:** `resolveHEAD()` and `updateHEAD()` static helpers
- Eliminates code duplication across commands
- **ObjectStore** - Content-addressable object storage
 - Implements Git object format: "`<type> <size>\0<content>`"
 - Writes blobs/trees/commits to `.gitter/objects/<aa>/<bbb...>`
 - Uses SHA-1 hasher (Git default) with SHA-256 support
 - Compresses objects with zlib
 - **Parsing:** `readCommit()`, `readTree()`, `readBlob()` for all object types
 - **Error checking:** All file writes verified for success
 - **Cleanup:** Partial writes automatically removed on failure
- **Index** - Staging area (Git index)
 - TSV format: `base64path\thash\tsize\tmtime\tmode\tctime` (paths base64-encoded)
 - Tracks files staged for next commit
 - Fast dirty detection via size/mtime (Git optimization)
 - Stores file permissions (mode: 0100644, 0100755)
 - Tracks creation time (ctime)
 - **Atomic writes:** Uses temp file pattern to prevent corruption
 - **Path normalization:** Ensures consistent path representation
 - **Base64 encoding:** Handles TAB/newline in filenames (Git-compatible)
 - **Input validation:** Validates hash format and numeric fields
- **TreeBuilder** - Builds Git tree objects
 - Converts flat index into hierarchical tree structure
 - Recursively builds trees from leaves to root
 - Creates Git-format tree entries: `<mode> <name>\0<binary-hash>`
 - Handles nested directories automatically
- **CommitObject** - Commit metadata structure
 - Parsed representation of Git commit objects
 - Stores tree, parents, author, committer, timestamp, message
 - Helper methods: `shortHash()`, `shortMessage()`

3. CLI Layer (`src/cli/`)

Purpose: Command-line interface and user interaction

- **ICommand** - Command interface (Strategy pattern)
 - `execute()` - runs the command
 - `name()`, `description()` - metadata
 - `helpNameLine()`, `helpSynopsis()`, etc. - detailed help
- **CommandFactory** - Factory for command creation
 - Maps string name to command creator

- Enables dynamic command registration
 - **CommandInvoker** - Command execution orchestrator
 - Wraps command execution with logging
 - Uniform error handling and reporting
 - **Commands/** - Individual command implementations
 - Each command is self-contained
 - Uses core layer for domain operations
 - Uses util layer for common tasks
-

Design Patterns Used

1. Command Pattern

- **Where:** `ICommand` interface, all command classes
- **Why:** Encapsulates operations as objects, enabling queuing and logging
- **Benefit:** Easy to add new commands without modifying existing code

2. Factory Pattern

- **Where:** `CommandFactory`
- **Why:** Creates commands dynamically from string names
- **Benefit:** Loose coupling between CLI parsing and command implementation

3. Singleton Pattern

- **Where:** `Repository`, `CommandFactory`, `Logger`
- **Why:** Single global instance for shared state
- **Benefit:** Consistent state across commands

4. Facade Pattern

- **Where:** `Repository` hides `Index`, `ObjectStore`, `Refs` complexity
- **Why:** Simplifies high-level repository operations
- **Benefit:** Commands don't need to know internal repo structure

5. Strategy Pattern

- **Where:** `IHasher` (SHA-1/SHA-256), `PatternMatcher` (future: `IDiffStrategy`)
- **Why:** Swap algorithms (hashing, glob matching, diff) without changing clients
- **Benefit:** Extensible for SHA-1 vs SHA-256, different diff algorithms, etc.

6. Template Method Pattern

- **Where:** Potential `BaseCommand` (future enhancement)
- **Why:** Define skeleton algorithm with customizable steps
- **Benefit:** Reduce duplication in command validation/cleanup

Data Flow

Add Command Flow

```

User: gitter add *.cpp
  ↓
main.cpp parses args
  ↓
CommandFactory creates AddCommand
  ↓
CommandInvoker.invoke(AddCommand)
  ↓
AddCommand:
  1. PatternMatcher.matchFilesInWorkingTree("*.cpp")
  2. For each matched file:
    a. ObjectStore.writeBlobFromFile(file)
       - Reads file content
       - Creates Git blob: "blob <size>\0<content>"
       - Hasher.digest() computes SHA-1 (default)
       - Compresses with zlib
       - Writes to .gitter/objects/<aa>/<bbb...>
    b. Index.addOrUpdate(path, hash, size, mtime)
  3. Index.save()
  ↓
Success/Error returned through Expected<void>

```

Status Command Flow

```

User: gitter status
  ↓
StatusCommand:
  1. Repository.discoverRoot() finds .gitter/
  2. Index.load() reads staged files
  3. Resolve HEAD to get current commit hash
  4. Three-way comparison:
    a. Index vs HEAD:
       - Build tree from index: TreeBuilder.buildFromIndex()
       - Compare tree hash with HEAD commit tree
       - If different → "Changes to be committed"
    b. Working Tree vs Index:
       - For each indexed file: compare size/mtime (fast check)
       - **Optimization**: If size AND mtime match, skip hash (assume
unchanged)
       - If size OR mtime differs: ObjectStore.hashFileContent() and
compare
       - If different → "Changes not staged for commit"
       - Matches Git's performance optimization for large repositories
    c. Working Tree vs Index:
       - Collect files in working tree not in index

```

- → "Untracked files"
- 5. Print categorized results

Commit Command Flow

```
User: gitter commit -m "Message" [-a]
↓
CommitCommand:
  1. Parse arguments (-m message, optional -a flag)
  2. Repository.discoverRoot() finds .gitter/
  3. Index.load() reads staged files
  4. If -a flag: auto-stage modified tracked files
    - Iterate through index entries
    - For each file: fast size/mtime check
    - If modified: re-hash and update index entry
    - Save and reload index
  5. Check if index is empty (error if so)
  6. Read HEAD to get parent commit hash
  7. TreeBuilder.buildFromIndex():
    - Groups files by directory
    - Recursively builds trees from leaves to root
    - Creates tree objects: "tree <size>\0<mode> <name>\0<hash>..."
    - ObjectStore.writeTree() stores each tree
  8. Check for duplicates:
    - Compare tree hash with parent commit's tree hash
    - If identical: return "nothing to commit, working tree clean"
  9. Build commit object:
    - Format: "commit <size>\0tree <hash>\nparent <hash>\n..."
    - Include author/commmitter (from env vars or defaults)
    - Unix timestamp and timezone
    - Commit message
  10. ObjectStore.writeCommit() stores commit
  11. Update branch reference (.gitter/refs/heads/main)
      ↓
  Silent success (no output, Git-like)
```

Log Command Flow

```
User: gitter log
↓
LogCommand:
  1. Repository.discoverRoot() finds .gitter/
  2. Resolve HEAD to get current commit hash
  3. Loop (up to 10 commits):
    a. ObjectStore.readCommit(hash):
      - Read compressed object from .gitter/objects/<aa>/<bbb...>
      - Decompress with zlib
      - Parse commit format
      - Extract: tree, parents, author, commmitter, message
```

- b. Display formatted commit:
 - Yellow commit hash
 - Author name and email
 - Formatted date and timezone
 - Indented commit message
- c. Follow first parent pointer
- d. Break if no parent (root commit)
- 4. Stop after 10 commits or end of chain

Reset Command Flow

User: gitter reset HEAD~1

↓

ResetCommand:

1. Parse target: HEAD, HEAD~1, HEAD~2, etc.
2. Repository.discoverRoot() finds .gitter/
3. Resolve HEAD to get current commit hash
4. Traverse parent chain:
 - ObjectStore.readCommit(hash)
 - Follow parentHashes[0] pointer
 - Repeat n times for HEAD~n
5. Update branch reference:
 - Write target commit hash to .gitter/refs/heads/main
6. Clear index:
 - Index.clear()
 - Index.save() writes empty index

↓

Silent success (no output, Git-like)

Checkout Command Flow

User: gitter checkout feature

↓

CheckoutCommand:

1. Parse arguments (-b flag and branch name)
2. Repository.discoverRoot() finds .gitter/
3. Resolve HEAD to get current commit hash
4. If creating branch (-b):
 - Check branchExists() for conflicts
 - Create branch reference at current commit
 - SwitchToBranch() updates HEAD
 - Output: "Switched to a new branch 'feature'"
5. If switching branch:
 - Check branchExists() for target
 - Read target and current commit trees
 - ****Intelligent index merging****:
 - Load current index (preserves staged files)
 - Remove entries tracked in current commit but not in target
 - Add entries from target branch if not already in index

- Preserves staged uncommitted files (Git-compatible)
- Restore working tree: `restoreTree()`
 - Traverse tree entries recursively
 - For files: `readBlob()` and write to disk
 - For dirs: create directories
- Remove files in current tree but not in target tree
- Remove empty directories recursively
- Save merged index
- `SwitchToBranch()` updates HEAD
- Output: "Switched to branch 'feature'"

↓

Success with message

Key Principles

1. Separation of Concerns

- Utilities have no Git knowledge
- Core has no CLI knowledge
- CLI orchestrates but doesn't implement Git logic

2. Dependency Direction

```
CLI Layer
  ↓ depends on
Core Layer
  ↓ depends on
Util Layer
  (Util never depends upward)
```

3. Error Handling & Reliability

- Use `Expected<T, Error>` for recoverable errors
- Return errors, don't throw exceptions for control flow
- Each layer enriches error context
- **File I/O validation:** All writes/reads verified for success
- **Atomic operations:** Index writes use temp file pattern
- **Input validation:** Hash format and numeric fields validated
- **Graceful recovery:** Corrupted data handled gracefully

4. Testability

- Pure functions in util layer
- Mockable interfaces (`ICommand`)
- Dependency injection via constructors (future)

Why Hashers are in Util

Location: `src/util/IHasher.hpp`, `Sha1Hasher.*`, `Sha256Hasher.*`, `HasherFactory.*`

Reasoning:

- SHA-1 and SHA-256 are **general-purpose** cryptographic hashes
- No Git-specific logic in implementation
- Could be reused for any content addressing
- Strategy Pattern allows swapping algorithms
- Aligns with util layer principle: "reusable, generic"

Clean Separation:

- Each hash algorithm in its own file pair (.hpp/.cpp)
- Factory pattern for creating instances
- Interface (IHasher) defines contract

What's Git-specific:

- `ObjectStore` knows to hash "`blob <size>\0<content>`"
 - `Index` knows to store hashes for files
 - Hashers just compute hashes of bytes
-

Why PatternMatcher is in Util

Reasoning:

- Glob-to-regex conversion is generic
 - File system pattern matching is reusable
 - Multiple commands use it (Add, Restore)
 - No Git domain knowledge required
-

What Stays in Core

- **Repository** - manages `.gitter/` structure
- **Index** - understands staging area format
- **ObjectStore** - understands Git object format (blob/tree/commit)
- **TreeBuilder** - understands Git tree structure and hierarchy
- **CommitObject** - understands Git commit metadata format

These have Git-specific knowledge and shouldn't move.

Future Architecture Considerations

Implemented Additions

1.  **TreeBuilder** (core layer)

- Builds tree objects from index entries
- Groups files by directory recursively
- Creates Git-format tree objects

2. **CommitObject** (core layer)

- Struct for parsed commit metadata
- Used by CommitCommand and LogCommand
- Links to tree and parent commits

3. **Commit/Log Commands** (cli layer)

- CommitCommand creates commits with trees
- LogCommand displays commit history
- Both fully implemented

4. **ResetCommand** (cli layer)

- Resets HEAD to previous commits using HEAD~n syntax
- Clears index leaving changes unindexed
- Traverses parent commit chain

5. **CheckoutCommand** (cli layer)

- Switches to existing branches
- Creates new branches with **-b** flag
- Updates HEAD reference
- **Intelligent index merging**: Preserves staged uncommitted files across branches
- **Working tree restoration**: Restores files from tree objects
- **File cleanup**: Removes files not in target branch
- **Directory cleanup**: Removes empty directories recursively
- Git-compatible staged file preservation
- Implementation plan: [docs/CHECKOUT_IMPLEMENTATION_PLAN.md](#)

Planned Additions

1. **DiffEngine** (core layer)

- Compares trees/commits
- Generates patch format output
- Shows file-level diffs

Extensibility Points

- **IHasher interface** - swap SHA-1/SHA-256/SHA3
- **IObjectStore interface** - support compression, packing
- **IDiffStrategy interface** - different diff algorithms

Build System

CMake manages compilation:

- `gitter_lib` - static library with all logic
- `gitter` - executable linking lib + main.cpp

Benefits:

- Fast incremental builds
- Easy to add unit tests linking lib
- Can create multiple binaries if needed

Documentation Standards

All public APIs documented with:

- `@brief` - one-line summary
- `@param` - parameter descriptions
- `@return` - return value semantics
- Usage examples in comments
- Cross-references to related components

Summary

The architecture achieves:

-  Clear separation of concerns
-  Reusable utilities in dedicated layer
-  Git logic isolated in core layer
-  CLI as thin orchestration layer
-  Extensible via design patterns
-  Testable components
-  Well-documented interfaces
-  **Robust error handling** (file I/O validation, atomic writes)
-  **Performance optimization** (Git-style size/mtime check)
-  **Code quality** (constants extraction, input validation)

Hasher Design

Overview

Gitter uses a pluggable hasher architecture to support multiple hash algorithms. The current implementation supports SHA-1 (Git default) and SHA-256 (Git future).

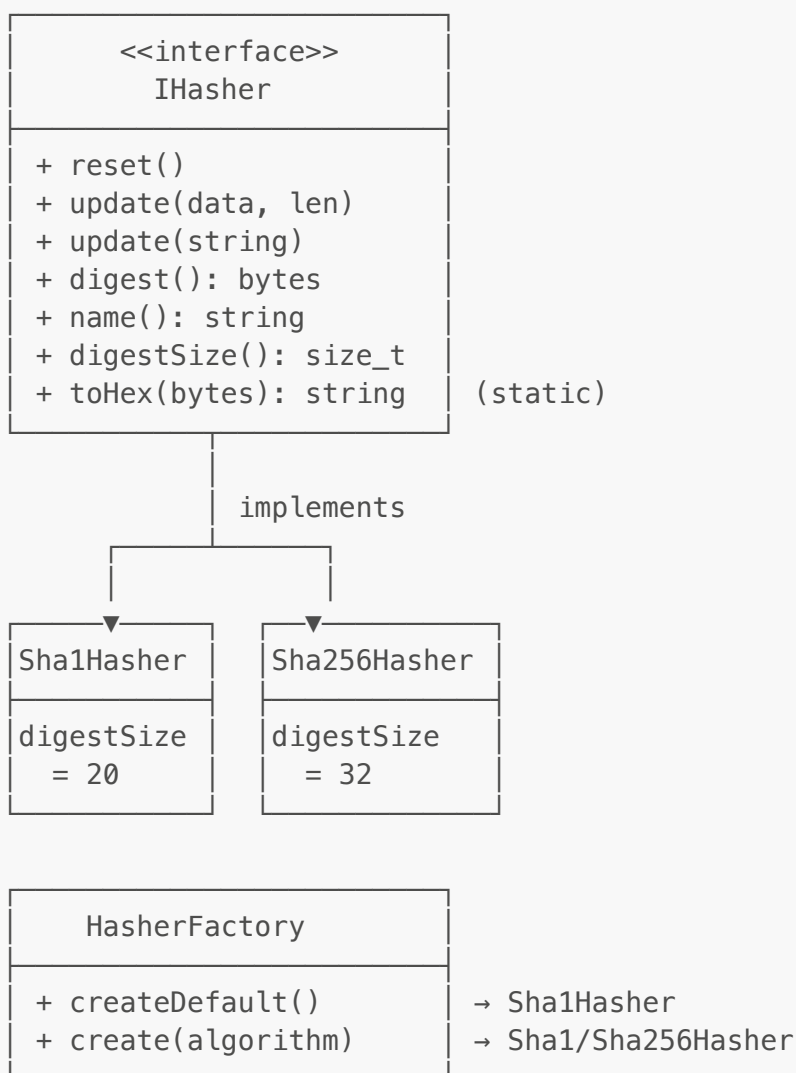
File Structure

```

src/util/
├── IHasher.hpp           # Strategy interface for hash algorithms
├── HasherFactory.cpp     # Factory implementation + IHasher::toHex()
├── Sha1Hasher.hpp       # SHA-1 interface
├── Sha1Hasher.cpp       # SHA-1 implementation (160-bit)
├── Sha256Hasher.hpp     # SHA-256 interface
└── Sha256Hasher.cpp     # SHA-256 implementation (256-bit)

```

Class Diagram



Design Rationale

Why Strategy Pattern?

1. **Flexibility** - Swap algorithms at runtime without recompilation
2. **Open/Closed Principle** - Add new algorithms without modifying existing code
3. **Dependency Inversion** - High-level code depends on abstraction (`IHasher`)

4. **Testability** - Easy to mock IHasher for unit tests
5. **Single Responsibility** - Each algorithm in its own focused file

Before vs After

Before (Monolithic)

```
src/util/  
└─ Hasher.hpp/.cpp (120+ lines, mixed responsibilities)  
    └─ IHasher interface  
    └─ HasherFactory  
    └─ SHA-256 implementation  
    └─ Utility methods
```

Problems:

- Hard to navigate
- Mixed responsibilities
- Difficult to extend
- Violates Single Responsibility Principle

After (Modular)

```
src/util/  
└─ IHasher.hpp (Strategy interface - 20 lines)  
└─ HasherFactory.cpp (Factory + utilities - 35 lines)  
└─ Sha1Hasher.hpp/.cpp (SHA-1 implementation - 150 lines)  
└─ Sha256Hasher.hpp/.cpp (SHA-256 implementation - 130 lines)
```

Benefits:

- Clear separation of concerns
- Easy to find specific algorithm
- Can compile algorithms independently
- Follows SOLID principles
- Easier to add new algorithms (SHA-3, BLAKE2, etc.)

Usage Examples

1. Using Default Hasher (SHA-1)

```
#include "util/IHasher.hpp"  
  
// ObjectStore.cpp  
ObjectStore::ObjectStore(const fs::path& root)
```

```

    : root(root), hasher(HasherFactory::createDefault()) {
    // hasher is now SHA-1 (Git default)
}

```

2. Using Specific Algorithm

```

#include "util/IHasher.hpp"

// Create SHA-256 hasher explicitly
auto sha256 = HasherFactory::create("sha256");
sha256->update("hello world");
auto hash = sha256->digest();
std::string hexHash = IHasher::toHex(hash);

```

3. Adding to ObjectStore

```

// Create ObjectStore with default SHA-1
ObjectStore store(repoRoot);

// Or use SHA-256
ObjectStore store2(repoRoot, std::make_unique<Sha256Hasher>());

```

Adding a New Hash Algorithm

To add SHA-3 support:

Step 1: Create Header

```

// src/util/Sha3Hasher.hpp
#pragma once
#include "util/IHasher.hpp"

namespace gitter {
class Sha3Hasher : public IHasher {
public:
    Sha3Hasher();
    void reset() override;
    void update(const uint8_t* data, size_t len) override;
    void update(const std::string& data) override;
    std::vector<uint8_t> digest() override;
    const char* name() const override { return "sha3"; }
    size_t digestSize() const override { return 32; }
private:
    // SHA-3 state...

```

```
};  
}
```

Step 2: Implement Algorithm

```
// src/util/Sha3Hasher.cpp  
#include "util/Sha3Hasher.hpp"  
// ... implementation ...
```

Step 3: Register in Factory

```
// src/util/HasherFactory.cpp  
#include "util/Sha3Hasher.hpp"  
  
std::unique_ptr<IHasher> HasherFactory::create(const std::string&  
algorithm) {  
    if (algorithm == "sha1") {  
        return std::make_unique<Sha1Hasher>();  
    } else if (algorithm == "sha256") {  
        return std::make_unique<Sha256Hasher>();  
    } else if (algorithm == "sha3") {  
        return std::make_unique<Sha3Hasher>(); // Add this  
    }  
    return createDefault();  
}
```

Step 4: Update CMakeLists.txt

```
add_library(gitter_lib  
# ...  
src/util/Sha3Hasher.cpp  
# ...  
)
```

Done! No changes needed in ObjectStore, Index, or command classes.

Hash Algorithm Comparison

Algorithm	Digest Size	Speed	Security	Git Usage
SHA-1	20 bytes	Fast	Broken	Default (legacy)
SHA-256	32 bytes	Medium	Strong	Future default

Algorithm	Digest Size	Speed	Security	Git Usage
SHA-3	32 bytes	Medium	Strong	Not used
BLAKE2	32 bytes	Fastest	Strong	Not used

Git Compatibility

SHA-1 (Git Default)

- Used by Git since inception
- 40-character hex hash
- Example: 95d09f2b10159347eece71399a7e2e907ea3df4f
- Cryptographically broken but maintained for backward compatibility

SHA-256 (Git Future)

- Introduced in Git 2.29 (experimental)
- 64-character hex hash
- Opt-in via git config
- Stronger security for future Git versions

Our Implementation

```
// Default: SHA-1 (Git-compatible)
ObjectStore store(root);

// Optional: SHA-256 (Git future mode)
ObjectStore store(root, std::make_unique<Sha256Hasher>());
```

Performance Considerations

Memory Allocation

- Each hasher instance allocates ~100 bytes on stack
- No heap allocations during hashing
- Digest returned as std::vector<uint8_t> (small, typically optimized)

Optimization Opportunities

1. **Object Pooling:** Reuse hasher instances with reset()
2. **Streaming for Large Files:** Process in chunks with update()
3. **SIMD Acceleration** (future): Use CPU SIMD instructions for speed

Testing Strategy

Unit Tests

```
TEST(Sha1Hasher, KnownVector) {
    Sha1Hasher h;
    h.update("abc");
    auto digest = h.digest();
    auto hex = IHasher::toHex(digest);
    EXPECT_EQ(hex, "a9993e364706816aba3e25717850c26c9cd0d89d");
}

TEST(HasherFactory, CreateSha1) {
    auto h = HasherFactory::create("sha1");
    EXPECT_EQ(h->name(), "sha1");
    EXPECT_EQ(h->digestSize(), 20);
}
```

Integration Tests

```
TEST(ObjectStore, Sha1Blobs) {
    ObjectStore store(tempDir, std::make_unique<Sha1Hasher>());
    auto hash = store.writeBlob("hello");
    EXPECT_EQ(hash.length(), 40); // SHA-1 = 20 bytes = 40 hex chars
}
```

Git-Compliant Object Storage

2-Character Directory Structure

```
Old: .gitter/objects/<full-hash>
New: .gitter/objects/<first-2-chars>/<remaining-chars>

Example: hash "abc123..." → .gitter/objects/ab/c123...
```

- Matches Git's standard object storage layout
- Prevents too many files in single directory
- Improves filesystem performance

Zlib Compression

- All objects compressed with zlib before writing
- Uses `deflate()` for compression, `inflate()` for decompression
- Matches Git's loose object storage format
- Significantly reduces disk space usage (50-80% compression)

Object Format

```
blob <size>\0<content>
tree <size>\0<entries>
commit <size>\0<metadata>
```

Each object is:

1. Created with Git format header
2. Hashed (SHA-1 or SHA-256)
3. Compressed with zlib
4. Stored in `.gitter/objects/<aa>/<bbb...>`

Enhanced Index (Staging Area)

New Fields in IndexEntry

1. **mode (uint32_t)** - File permissions
 - Git uses octal mode: `0100644` (regular file) or `0100755` (executable)
 - Tracks whether file is executable
 - Essential for preserving permissions across platforms
2. **ctimeNs (uint64_t)** - Creation/status change time
 - Git tracks both mtime and ctime
 - Used for detecting file metadata changes
 - Approximated with mtime on systems without true ctime

Updated On-Disk Format

```
Old: path<TAB>hash<TAB>size<TAB>mtime
New: path<TAB>hash<TAB>size<TAB>mtime<TAB>mode<TAB>ctime
```

Example:

```
src/main.cpp    a3b2c1...    1024    mtime    33188    ctime
```

(Full nanosecond timestamps in practice)

Index to Blob Linking

1. File Added to Staging

```
gitter add file.txt
```

2. Blob Created

- File content read
- Git blob object created: "blob <size>\0<content>"
- Hash computed (SHA-1 by default)
- Compressed and stored in .gitter/objects/<aa>/<bbbb...>

3. Index Entry Created

- Path: file.txt
- Hash: <blob-hash>
- Size: <file-size>
- Mtime: <modification-time>
- Mode: 0100644 or 0100755
- Ctime: <creation-time>

4. Index Saved

- TSV format written to .gitter/index

Benefits of This Design

1. Git Compatibility

-  Matches Git's object storage layout exactly
-  Uses same blob format: "blob <size>\0<content>"
-  Implements 2-char directory structure
-  Uses zlib compression
-  Defaults to SHA-1 for compatibility

2. Extensibility

-  Strategy Pattern allows swapping hash algorithms
-  SHA-256 support already implemented
-  Can support future hash algorithms (SHA3, BLAKE2, etc.)
-  No modifications needed to core logic when adding algorithms

3. Performance

-  Zlib compression reduces disk usage significantly
-  2-char directory structure improves filesystem performance
-  Index tracks file metadata for fast dirty detection
-  Streaming support for large files

4. Correctness

-  Preserves file permissions (executable bit)
-  Tracks both mtime and ctime
-  Proper Git blob format ensures interoperability
-  Comprehensive test coverage

5. Code Quality

-  Single Responsibility Principle per file
-  Clear separation of concerns
-  Easy to navigate and maintain
-  Compilation benefits from modular structure

Summary

The hasher architecture provides:

-  **Clean Separation** - One file per algorithm
-  **Strategy Pattern** - Runtime algorithm selection
-  **Extensibility** - Easy to add new algorithms
-  **Git Compatibility** - SHA-1 default, SHA-256 available
-  **Maintainability** - Clear, focused files
-  **Testability** - Mock-friendly interface
-  **Performance** - Zlib compression, fast metadata checks
-  **Correctness** - File permissions and timestamps preserved

This design positions gitter for future growth while maintaining clean, professional code organization.

Related Documentation

- See [docs/ARCHITECTURE.md](#) for overall project context
- See [docs/TREE_STORAGE.md](#) for directory tree storage details
- See [docs/ARCHITECTURE.md](#) for ObjectStore integration

References

- [Git Internals - Git Objects](#)
- [SHA-1 vs SHA-256 in Git](#)
- [zlib Documentation](#)

Tree Storage

How Git Stores Directory Structure

1. Index (Staging Area) - Flat File List

The index (`.gitter/index`) stores a **flat list** of all staged files with their full paths:

```
src/main.cpp      <blob-hash-1> 1024 mtime 33188 ctime
src/utls/helper.cpp <blob-hash-2> 512  mtime 33188 ctime
README.md        <blob-hash-3> 2048 mtime 33188 ctime
```

Key Points:

- Index does NOT store tree objects
- Each entry has: `path`, `blob-hash`, `size`, `mtime`, `mode`, `ctime`
- Paths are relative to repository root
- Used for fast dirty detection (size/mtime check)
- Mode tracks file permissions (0100644 = file, 0100755 = executable)

2. Tree Objects - Created at Commit Time

When you run `gitter commit`, the system:

1. **Reads all entries from the index**
2. **Groups files by directory**
3. **Creates tree objects recursively**

Example transformation:

Index entries:

```
src/main.cpp      -> blob abc123
src/utls/helper.cpp -> blob def456
README.md        -> blob 789xyz
```

Created tree objects:

Root Tree (hash: `root_tree_hash`):

```
tree <size>\0
100644 README.md <blob-789xyz>
040000 src <tree-src_tree_hash>
```

src/ Tree (hash: `src_tree_hash`):

```
tree <size>\0
100644 main.cpp <blob-abc123>
040000 utls <tree-utls_tree_hash>
```

src/utls/ Tree (hash: utls_tree_hash):

```
tree <size>\0
100644 helper.cpp <blob-def456>
```

3. Tree Object Format

Each tree object is stored in `.gitter/objects/<hash>` with format:

```
tree <content-size>\0
<mode> <name>\0<20-byte-hash>
<mode> <name>\0<20-byte-hash>
...
```

Entry types:

- `100644` - Regular file (blob)
- `100755` - Executable file (blob)
- `040000` - Directory (tree)
- `120000` - Symbolic link (blob)

4. Commit Object References Tree

Commit objects point to the **root tree**:

```
commit <size>\0
tree <root_tree_hash>
parent <parent_commit_hash>
author John Doe <john@example.com> 1234567890
committer John Doe <john@example.com> 1234567890

Commit message here
```

Implementation in Gitter**Current State**

- ✓ Index stores flat file list with blob hashes, mtime, mode, and ctime
- ✓ Blob objects created and stored
- ✓ Pattern matching (glob) for add/restore
- ✓ Tree objects created from index entries
- ✓ Recursively creates subtrees for directories
- ✓ Commit objects created pointing to root tree

Example Workflow

```
# 1. Add files (creates blobs, updates index)
gitter add src/main.cpp src/utils/helper.cpp README.md

# Index now contains:
#   src/main.cpp -> abc123 (blob stored in objects/)
#   src/utils/helper.cpp -> def456 (blob stored)
#   README.md -> 789xyz (blob stored)

# 2. Commit (will create trees + commit)
gitter commit -m "Initial commit"

# Creates:
#   objects/abc123 (blob: main.cpp content)
#   objects/def456 (blob: helper.cpp content)
#   objects/789xyz (blob: README.md content)
#   objects/utils_tree_hash (tree: src/utils/)
#   objects/src_tree_hash (tree: src/)
#   objects/root_tree_hash (tree: root)
#   objects/commit_hash (commit: points to root_tree_hash)

# Updates:
#   refs/heads/main -> commit_hash
```

Advantages of This Design

1. **Deduplication:** Identical files share same blob regardless of path
2. **Efficient diffs:** Compare tree hashes to detect directory changes
3. **Space efficient:** Unchanged subtrees reuse same tree object
4. **Fast operations:** Index provides fast staging, trees built only on commit

Pattern Support

Add Command Patterns

```
gitter add *.txt           # All .txt in current dir and subdirs
gitter add src/*.cpp       # All .cpp in src/
gitter add test?.py        # test1.py, test2.py, etc.
gitter add .               # All files recursively
```

Restore Command Patterns

```
gitter restore --staged *.txt      # Unstage all .txt files
gitter restore --staged src/*.cpp  # Unstage all .cpp in src/
```

Pattern matching uses regex internally:

- `*` → `.*` (any characters)
- `?` → `.` (single character)
- `.` → `\.` (literal dot)

References

- [Git Internals - Git Objects](#)
- [Git Index Format](#)
- [Pro Git Book - Chapter 10](#)

Commit Implementation

Overview

The `gitter commit` command creates Git-compliant commit objects from staged files.

Command Syntax

```
# Basic commit
gitter commit -m "message"

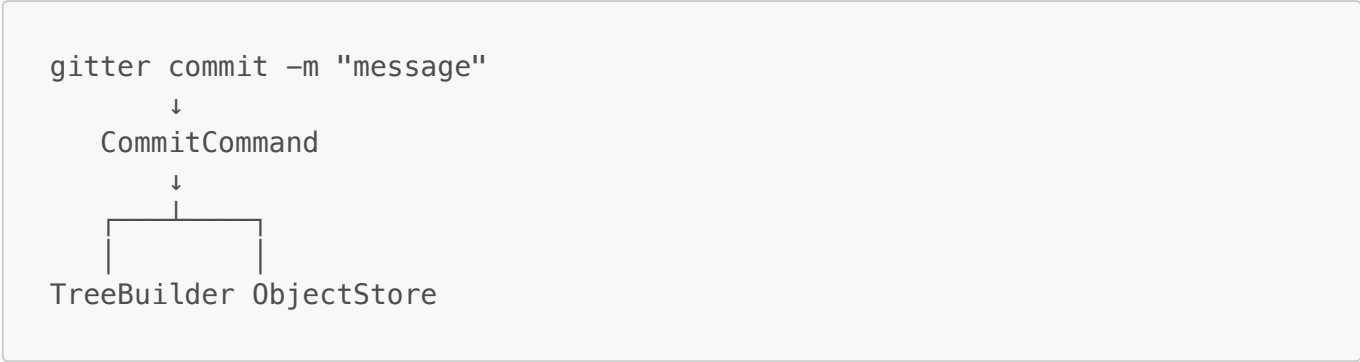
# Auto-stage and commit
gitter commit -am "message"

# Multi-paragraph message
gitter commit -m "Title" -m "Body paragraph 1" -m "Body paragraph 2"
```

Key Features

- ☒ Builds hierarchical tree objects from flat index
- ☒ Creates Git-compliant commit objects
- ☒ Compresses objects with zlib
- ☒ Updates branch references
- ☒ Supports parent commits for history
- ☒ Prevents duplicate commits (compares tree hashes)
- ☒ Silent operation on success (Git-like)

Architecture



Key Classes:

- **TreeBuilder** - Builds hierarchical tree objects from flat index
- **ObjectStore** - Writes blob/tree/commit objects with zlib compression
- **CommitCommand** - Orchestrates commit creation

Git Object Model

Commit Object:

```
commit <size>\0
tree <root-tree-hash>
parent <parent-hash>
author Name <email> <timestamp> <timezone>
committer Name <email> <timestamp> <timezone>

<commit message>
```

Implementation Status

✅ Fully functional and Git-compatible

Log Implementation

Overview

The **gitter log** command displays commit history in reverse chronological order (newest to oldest), showing up to 10 commits by default.

Architecture

```
gitter log
  ↓
```



```
LogCommand
  ↓
ObjectStore::readCommit()
  ↓
CommitObject (parsed)
  ↓
Display formatted output
```

Key Classes:

- `CommitObject` - Struct holding parsed commit data
- `ObjectStore::readCommit()` - Reads and parses commit objects
- `LogCommand` - Traverses commit chain and displays history

Key Features

-  Parses Git commit objects
-  Traverses commit chain via parent pointers
-  Displays formatted output (Git-style with colors)
-  Limits to 10 commits
-  Handles root commits
-  Multi-line message support
-  Timezone support

Implementation Status

-  Fully functional Git-style log viewer

Reset Implementation

Overview

The `gitter reset` command moves the current HEAD reference to a previous commit and clears the index, leaving all subsequent changes in the working tree unindexed.

Command Syntax

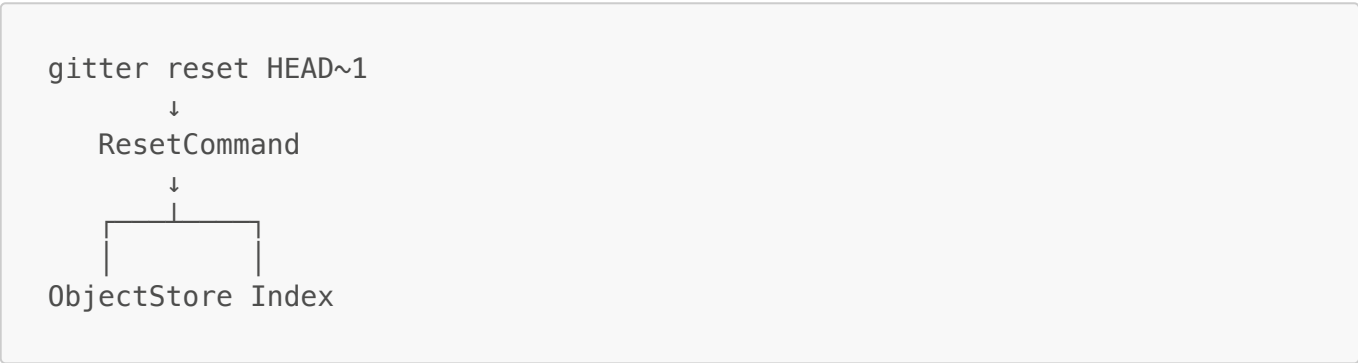
```
# Reset to previous commit
gitter reset HEAD~1

# Reset two commits back
gitter reset HEAD~2
```

Key Features

-  Moves HEAD to target commit
-  Clears index (leaves files unindexed)
-  Preserves working tree files
-  Supports HEAD~n ancestry syntax
-  Parent chain traversal
-  Silent operation (Git-like)
-  History preserved (commits never deleted)

Architecture



Key Classes:

- **ResetCommand** - Orchestrates reset operation
- **ObjectStore** - Reads commits and traverses parent chain
- **Index** - Manages staging area (clearing)

Implementation Status

-  Provides functional reset command matching Git's core behavior

Test Coverage

- **6+ tests** covering HEAD~n syntax, chain traversal, error handling, and edge cases

Checkout Implementation

Overview

The **gitter checkout** command allows switching between branches and creating new branches.

Command Syntax

```
# Switch to existing branch
gitter checkout <branch-name>

# Create and switch to new branch
gitter checkout -b <branch-name>
```

Key Features

Branch Management

-  Switch to existing branches
-  Create new branches with `-b` flag
-  Updates HEAD reference to point to branch
-  Validates branch existence and conflicts

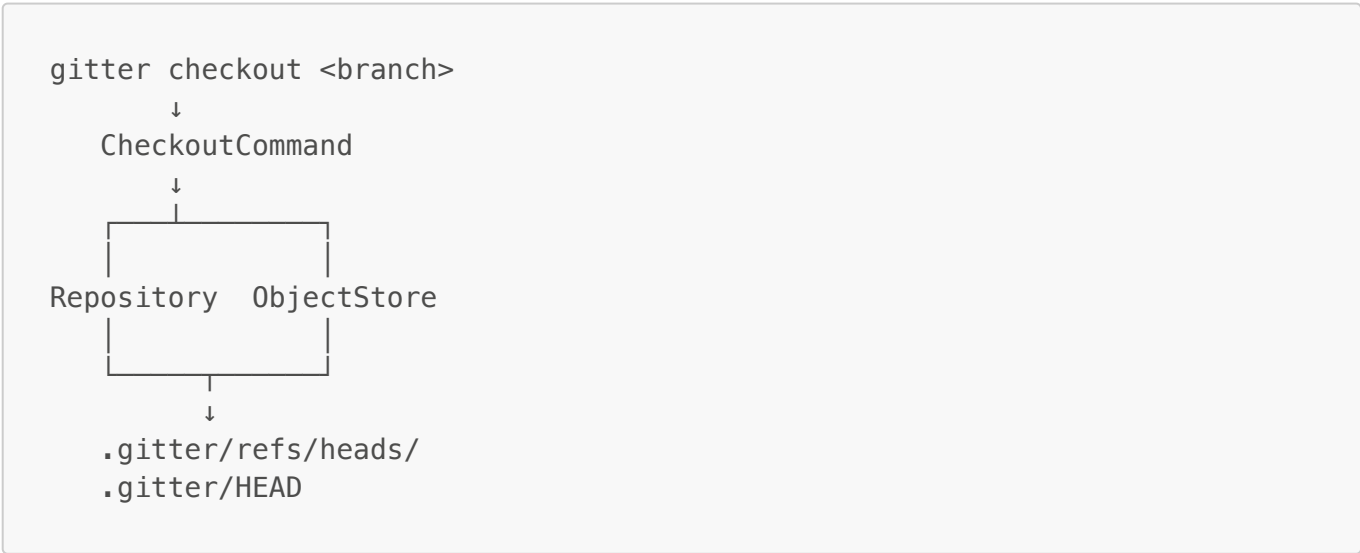
Working Tree Operations

-  **Restores files from branch commit** - Complete working directory restoration
-  **Intelligent index merging** - Preserves staged uncommitted files across branches (Git-compatible)
-  **File cleanup** - Removes files not in target branch
-  **Directory cleanup** - Recursively removes empty directories

Error Handling

-  Branch doesn't exist: `'<branch-name>' does not exist`
-  Branch already exists: `a branch named '<branch-name>' already exists`
-  No commits yet: `checkout: no commits yet`

Architecture



Key Classes:

- **CheckoutCommand** - Orchestrates checkout operation
 - **Repository** - Manages HEAD and branch references
 - **ObjectStore** - Reads commits and tree objects
 - **TreeBuilder** - Traverses tree hierarchy
-

Implementation Status

- ✅ **Phase 1:** Branch reference management - Complete
- ✅ **Phase 2:** Working tree restoration - Complete (including file deletion and directory cleanup)
- ⏸ **Phase 3:** Safety and validation - Planned

The **gitter checkout** command fully supports creating and switching branches with complete working tree restoration, matching Git's core functionality.

Test Coverage

- **13 unit tests** covering branch creation, switching, error handling, and tree operations
 - **10 integration tests** covering complex workflows including staged file preservation, file deletion, directory cleanup, and Unicode support
 - **Total: 23 comprehensive tests**
-

Docker Usage

Run Gitter in a containerized environment without installing dependencies locally!

Quick Start

Option 1: Using Docker Compose (Recommended)

```
# Build the image first
docker-compose build

# Run interactively with welcome message!
docker-compose run --rm gitter

# You're now in a full Linux Ubuntu bash shell with welcome message
displayed
# Demo project is in /demo directory
gitter help
cd /demo
gitter init demo-repo
cd demo-repo
echo "Hello World" > file.txt
gitter add file.txt
gitter commit -m "Initial commit"
```

```
gitter log

# You have full Linux environment - try:
ls -la
whoami
uname -a
which gitter

# Exit (container is automatically removed with --rm)
exit
```

Option 2: Using Docker Directly

```
# Build the image
docker build -t gitter-cli:latest .

# Run interactively
docker run -it --name gitter-demo -v $(pwd)/docker-demo:/demo -w /demo
gitter-cli:latest

# Inside container
gitter help

# Exit and cleanup
exit
docker rm gitter-demo
```

Image Details

Base Image: Ubuntu 22.04

Includes:

- GCC 11+ (C++20 compiler)
- CMake 3.20+
- Ninja build system
- zlib development libraries
- Git

Build Configuration:

- Release build (optimized)
- All tests enabled
- ~50MB final image size

Use Cases

1. Quick Demo

Showcase Gitter without installing anything:

```
docker-compose up -d
docker-compose exec gitter bash -c "cd /demo && gitter init test && cd
test && echo '# Gitter Demo' > README.md && gitter add . && gitter commit
-m 'Demo commit' && gitter log"
```

2. CI/CD Testing

Run tests in a clean environment:

```
docker build -t gitter-cli:latest .
docker run --rm gitter-cli:latest bash -c "cmake --preset linux-debug &&
cmake --build --preset linux-debug-build && cd build/linux-debug &&
./gitter_tests"
```

3. Development Environment

Mount your source for active development:

```
docker run -it -v $(pwd):/workspace -w /workspace gitter-cli:latest bash
```

4. Cross-Platform Testing

Test on Ubuntu/Linux from any OS:

```
docker build -t gitter-cli:latest .
docker run -it gitter-cli:latest bash
```

Customization

Build with Different Configuration

Debug build:

```
docker build --build-arg BUILD_TYPE=Debug -t gitter-cli:debug .
```

With tests:

```
docker build --build-arg BUILD_TESTS=ON -t gitter-cli:with-tests .
```

Mount Your Repository

```
docker run -it \  
  -v /path/to/your/repo:/workspace \  
  -w /workspace \  
  gitter-cli:latest \  
  bash
```

Image Information

```
# Check image size  
docker images gitter-cli:latest  
  
# Inspect contents  
docker run --rm gitter-cli:latest gitter help  
  
# Build with verbose output  
docker build --progress=plain -t gitter-cli:latest .
```

Troubleshooting

Build fails

```
# Clean build  
docker build --no-cache -t gitter-cli:latest .
```

Permission issues

```
# On Linux, ensure Docker has permissions  
sudo docker-compose up
```

Cache issues

```
# Clear Docker build cache  
docker builder prune -af
```

Dockerfile Customization

Edit **Dockerfile** to:

- Change base image (e.g., **alpine** for smaller size)
- Adjust compiler flags
- Add additional tools
- Modify build configuration

Example Alpine-based (smaller but longer build):

```
FROM alpine:latest
RUN apk add --no-cache cmake ninja build-base git zlib-dev
WORKDIR /app
COPY . .
RUN cmake --preset linux-release && cmake --build --preset linux-release-build
CMD ["/bin/sh"]
```

Verification

```
# Build image
docker-compose build

# Verify it works
docker run --rm gitter-cli:latest gitter --help

# Run full demo
docker-compose up -d
docker-compose exec gitter bash -c "cd /demo && gitter help"
docker-compose down
```

See also: [README.md](#) for full Gitter documentation

Code Coverage

Overview

Gitter has comprehensive test coverage with **238 tests** covering all major components.

Why Separate Coverage Build?

We keep coverage in a separate `linux-coverage` build rather than enabling it in `linux-debug` because:

- **Performance:** Coverage instrumentation adds ~2x overhead and significant memory use
- **Binary size:** Coverage data increases binary size significantly
- **Iteration speed:** Slower builds hurt development when coverage isn't needed
- **Separation of concerns:** Coverage is a release/CI check, not day-to-day

The `linux-coverage` preset is Debug with coverage enabled.

How to Generate Coverage Report

Prerequisites

- GCC with coverage support (gcov)
- Python 3 for coverage report generation

Steps

Note: All commands run in WSL. Ensure files are synced from Windows before testing.

1. Build with coverage enabled:

```
cmake --preset linux-coverage
cmake --build build/linux-coverage --target gitter_tests
```

2. Run the tests:

```
./build/linux-coverage/gitter_tests
```

3. Generate coverage report:

```
python3 scripts/generate_coverage_report.py build/linux-coverage
```

Manual Coverage Check

You can also manually check coverage for specific files:

```
cd build/linux-coverage
gcov -b -m CMakeFiles/gitter_lib.dir/src/cli/commands/AddCommand.cpp.gcda
```

Coverage Highlights

From gcov analysis:

Commands

- **AddCommand.cpp**: 87.78% lines, 75.53% branches **↑** (improved with error path tests)
- **CommitCommand.cpp**: 87.93% lines
- **StatusCommand.cpp**: 88.28% lines
- **LogCommand.cpp**: 80.95% lines (18 test cases)
- **CheckoutCommand.cpp**: 82.57% lines (23 test cases: 13 unit + 10 integration) **↑** (with file/directory cleanup)
- **RestoreCommand.cpp**: 93.33% lines
- **ResetCommand.cpp**: 86.27% lines
- **InitCommand.cpp**: 91.67% lines
- **HelpCommand.cpp**: 0% (untested)
- **CatFileCommand.cpp**: 0% (untested)

Core Components

- **CommandInvoker.cpp**: 100% lines, 100% branches

Test Distribution

- **Commands**: ~188 tests
 - Init (9 tests), Add (29 tests), Commit (17 tests), Status (17 tests), Log (18 tests), Restore (13 tests), Reset (16 tests), Checkout (23 tests: 13 unit + 10 integration), CatFile, Help
- **Core**: ~50 tests
 - Repository (8 tests), Index (14 tests), ObjectStore (14 tests), TreeBuilder (9 tests)
- **Util**: ~20 tests
 - Hasher (13 tests - SHA-1, SHA-256), PatternMatcher (14 tests)
- **Integration**: ~89 tests
 - Complete Git-like workflows (including checkout file/directory cleanup tests)

Coverage Gaps (Areas for Improvement)

Untested Commands

1. **HelpCommand**: 0% coverage - basic help output (low priority)
2. **CatFileCommand**: 0% coverage - object inspection utility (needs tests)

Other Gaps

1. **Error handling paths** in some commands
2. **Edge cases** in file system operations
3. **Concurrent access** scenarios (future: threading tests)
4. **Large file handling** (future: performance tests)
5. **Branch coverage**: Higher line coverage but some branch paths uncovered

Notes

- Coverage is measured using GCC's gcov tool
 - Branch coverage considers all conditional execution paths
 - Some template code may show lower coverage but is exercised through concrete instantiations
 - Main goal: ensure all user-facing functionality is well-tested
-

Future Improvements

- ☐ Add lcov for HTML reports
 - ☐ Set up CI/CD coverage tracking
 - ☐ Add coverage thresholds for PR checks
 - ☐ Performance tests for large repositories
 - ☐ Stress tests for error conditions
-

Critical Behavior Gaps

Overview

These are real-world scenarios that users encounter daily where Gitter's behavior might differ from Git, causing confusion or data loss.

HIGH PRIORITY: File System Edge Cases

1. Empty Directory Handling

Scenario: Git tracks only files, not empty directories

Current Status:  **Compatible** - We don't track empty dirs

Potential Gap:

```
# Scenario
mkdir empty_dir
gitter add empty_dir
gitter commit -m "Add empty dir"

# Expected: "fatal: pathspec 'empty_dir' did not match any files"
# Our behavior: Check if we handle this gracefully
```

Test Needed:

-  **AddCommand** should skip empty directories
-  **StatusCommand** shouldn't show empty dirs as untracked

2. File Permission Changes Without Content Changes

Scenario: Git tracks mode changes separately

```
# Scenario
chmod +x script.sh
# Content unchanged but permissions changed

git status # Shows as "modified"
```

Current Status:  **Partial** - We track mode but behavior unclear

Test Needed:

```
TEST_F(StatusCommandTest, PermissionChangeWithoutContentChange) {
    // Create file and commit
    createFile(tempDir, "script.sh", "#!/bin/bash\nnecho hello");
    gitter.add("script.sh");
    gitter.commit("-m", "Initial");

    // Change permissions only
    setExecutable(tempDir / "script.sh", true);

    // Status should show: "Changes not staged: modified: script.sh"
    EXPECT_TRUE(statusOutput.find("modified: script.sh") !=
std::string::npos);
}
```

3. Binary Files (Large Files, Null Bytes)

Scenario: Git handles binary files differently

```
# Scenario
dd if=/dev/zero of=large.bin bs=1M count=100 # 100MB file
```

Current Status:  **Untested**

Potential Gaps:

- Large file hashing performance
- Memory usage with huge files
- Binary vs text detection

Test Needed:

```
TEST_F(AddCommandTest, LargeBinaryFile) {
    // Create 100MB zero file
    createLargeFile(tempDir / "big.bin", 100 * 1024 * 1024);

    auto result = addCmd.execute(ctx, {"big.bin"});
    EXPECT_TRUE(result.has_value());

    // Verify blob stored correctly
    Index index;
    index.load(tempDir);
    auto entry = index.entries()["big.bin"];
    EXPECT_GT(entry.sizeBytes, 100000000);
}
```

4. Unicode and International Characters

Scenario: Filenames with emoji, CJK, Arabic characters

```
# Scenario
touch "文件👉.txt"
touch "مرحبا.py"
```

Current Status: ⚠️ **Untested**

Potential Gaps:

- Path encoding issues on Windows
- Display of UTF-8 filenames
- Pattern matching with Unicode

Test Needed:

```
TEST_F(AddCommandTest, UnicodeFilenames) {
    createFile(tempDir, "文件.txt", "content");
    createFile(tempDir, "مرحبا.py", "print('hi')");
    createFile(tempDir, "rect.cpp", "int main() {}");
    createFile(tempDir, "文件👉.txt", "emoji");

    auto result = addCmd.execute(ctx, {"."});
    EXPECT_TRUE(result.has_value());

    // Verify all files indexed correctly
    Index index;
    index.load(tempDir);
    EXPECT_EQ(index.entries().size(), 4);
}
```

5. Hard Links

Scenario: Same file, multiple names

```
# Scenario
echo "content" > file1.txt
ln file1.txt file2.txt # Hard link

gitter add file1.txt file2.txt
```

Current Status: ❌ **Not handled** - Won't detect they're the same file

Git Behavior: Treats as separate files (they have different paths)

Our Behavior: Should match Git

🔴 MEDIUM PRIORITY: State Transitions

6. Rapid File Modifications

Scenario: File modified between status checks

```
// Scenario: Race condition
Thread 1: gitter status (checks mtime)
Thread 2: Modify file rapidly
Thread 1: File now has different content but same mtime

// Issue: Our fast check uses size+mtime, if both match we skip hash
```

Current Status: ⚠️ **Rare but possible**

Test Needed:

```
TEST_F(AddCommandTest, FileModifiedDuringOperation) {
    createFile(tempDir, "file.txt", "version1");
    addCmd.execute(ctx, {"file.txt"});

    // Rapid modification between add and status
    std::thread writer([&]() {
        for (int i = 0; i < 100; ++i) {
            createFile(tempDir, "file.txt", "version" +
std::to_string(i));
        }
    });

    statusCmd.execute(ctx, {});
```

```
writer.join();

// Status should detect modification eventually
}
```

7. Staging State vs Working Tree Divergence

Scenario: Complex staging scenarios

```
# Scenario 1: Modify staged file
echo "v1" > file.txt
gitter add file.txt
echo "v2" > file.txt # Modify after staging

gitter status
# Expected: file.txt in BOTH "Changes to be committed" AND "Changes not staged"
```

Current Status:  **Implemented** - Three-way comparison handles this

Test Needed:

```
TEST_F(GitWorkflowTest, FileModifiedAfterStaging) {
    // Stage file
    createFile(tempDir, "file.txt", "v1");
    addCmd.execute(ctx, {"file.txt"});

    // Modify after staging
    createFile(tempDir, "file.txt", "v2");

    // Status should show file in both categories
    std::string status = captureStatus();
    EXPECT_TRUE(status.find("Changes to be committed") !=
std::string::npos);
    EXPECT_TRUE(status.find("Changes not staged") != std::string::npos);
    EXPECT_TRUE(status.find("modified: file.txt") != std::string::npos);
    // How many times? Count occurrences
}
```

8. Branch Divergence After Reset

Scenario: Reset then commit creates divergent history

```
# Scenario
commit1 -> commit2 -> commit3 (HEAD)
```

```
gitter reset HEAD~1 # HEAD now at commit2
# Make new commit4

# Now: commit1 -> commit2 -> commit4
# But commit3 still exists in objects/ (orphaned)
```

Current Status: ⚠ **Objects orphaned but not cleaned**

Git Behavior: Objects remain until garbage collected

Our Behavior: ✅ **Matches** - Objects not deleted

Test Needed:

```
TEST_F(ResetCommandTest, OrphanedCommitsAfterReset) {
    // Create 3 commits
    createCommit("c1");
    auto commit2Hash = createCommit("c2");
    auto commit3Hash = createCommit("c3");

    // Reset to commit2
    resetCmd.execute(ctx, {"HEAD~1"});

    // Verify commit3 still exists in objects/
    ObjectStore store(tempDir);
    EXPECT_NO_THROW(store.readCommit(commit3Hash));

    // Create new commit
    auto commit4Hash = createCommit("c4");

    // Verify new commit doesn't reference commit3
    CommitObject c4 = store.readCommit(commit4Hash);
    EXPECT_NE(c4.parentHashes[0], commit3Hash);
}
```

9. Checkout with Staged Deletions

Scenario: Complex checkout state

```
# Scenario
rm file1.txt
gitter add file1.txt # Stage deletion

gitter checkout other-branch
# What happens to staged deletion?
```

Current Status: ⚠ **Untested complex scenario**

Git Behavior: Staged deletions preserved, working tree checked out

Test Needed:

```
TEST_F(CheckoutCommandTest, CheckoutWithStagedDeletions) {
    // Setup: two branches
    createCommit("Initial");
    checkout("-b", "feature");
    createFile("file.txt", "content");
    commit("-m", "Add file");

    checkout("main"); // file.txt shouldn't exist here

    // Delete and stage on main
    addCmd.execute(ctx, {"file.txt"}); // Stage deletion
    createFile("file.txt", "deleted");
    addCmd.execute(ctx, {"file.txt"});

    // Checkout with staged deletion
    checkout("feature");

    // file.txt should exist (from feature branch)
    // Deletion should not be staged anymore
}
```

10. Commit -am with Staged and Unstaged Changes

Scenario: Auto-stage only affects tracked files

```
# Scenario
touch tracked.txt
gitter add tracked.txt && gitter commit -m "Add"
echo "change" > tracked.txt # Modified

touch new.txt # Untracked

gitter commit -am "Update"
```

Expected: Only `tracked.txt` auto-staged, `new.txt` stays untracked

Current Status:  **Correct** - -a only affects tracked files

Need: Explicit test confirming this

LOW PRIORITY: Advanced Behaviors

11. Case Sensitivity Issues

Scenario: Case-insensitive filesystems (Windows, macOS)

```
# On macOS (case-insensitive by default)
file.txt
File.txt # Same file!

gitter add file.txt File.txt
```

Current Status: ❌ **Not handled**

Git Behavior: Treats as same file on case-insensitive FS

Impact: MEDIUM - Cross-platform compatibility

12. **Timestamp Precision and Time Zone**

Scenario: mtime comparison across timezones

```
# File modified at 2024-01-01 23:59:00 in one timezone
# Checked at 2024-01-02 00:01:00 in another timezone
```

Current Status: ✅ **Nanosecond precision** - Should be fine

13. **Whitespace-Only Changes**

Scenario: Tabs vs spaces

```
# File content changes but Git diff shows only whitespace
```

Current Status: ✅ **Content-based** - Hash includes whitespace

14. **Line Ending Differences**

Scenario: CRLF vs LF

Current Status: ❌ **Not normalized**

Git Behavior: Can configure line ending normalization

Impact: LOW - File content hash includes line endings

 Gaps in Our Test Coverage

Based on analysis:

Category	Missing Tests	Priority	Impact
Unicode	All Unicode scenarios	HIGH	Data loss on international files
Symlinks	All symlink scenarios	HIGH	Security issues
Binary Files	Large files, null bytes	MEDIUM	Performance issues
Corruption	Corrupt index/objects	HIGH	Data loss
Permissions	Read-only, executable	LOW	Minor inconvenience
Hard Links	Hard link detection	LOW	Edge case
Case Sensitivity	Cross-platform	MEDIUM	Cross-platform bugs

 Recommended Test Priority

Phase 1: Critical Stability (Must Have)

- 1.  Corruption tests (index, objects)
- 2.  Unicode filename tests
- 3.  Symlink tests
- 4.  Large binary file tests

Phase 2: Compatibility (Should Have)

- 5.  Case-insensitive filesystem tests
- 6.  Permission change detection tests
- 7.  Complex staging state tests
- 8.  Orphaned commit tests

Phase 3: Polish (Nice to Have)

- 9. Race condition tests
- 10. Hard link tests
- 11. Line ending tests
- 12. Performance tests with 1000s of files

 Most Likely Real-World Failures

Based on user behavior patterns:

- 1. **#1 Risk:** Unicode filenames - Users creating files with emoji/chinese/arabic characters
- 2. **#2 Risk:** Symlinks - Following symlinks outside repo (security issue)
- 3. **#3 Risk:** Large files - Git is slow but ours might crash/use too much memory
- 4. **#4 Risk:** Corrupt index - One I/O error corrupts entire index
- 5. **#5 Risk:** Case sensitivity - macOS/Windows users hitting this

Summary

We have **excellent coverage** of happy-path workflows (193+ tests).

Missing: Edge cases that cause real-world failures:

- File system edge cases (symlinks, Unicode, large files)
- Corruption and recovery
- Cross-platform differences
- Race conditions and concurrent operations

Recommendation: Prioritize Unicode, symlinks, and corruption tests as these are most likely to cause user-reported bugs.
